DAILY JAVA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Java – Beautifully Broken

ZGC wants to stop asking you for -Xmx, and IBM and Red Hat find 400 holes in Java's libraries

Issue #8 | Mon, 12 Oct 2026 | ⏱️ ~39 min read | Level 3/5

☕ **Today's two stories**

1. **ZGC is learning to choose its own heap size.** On 7 October, Oracle's Erik Österlund opened the pull request that implements JEP 545 (faster startup and warmup) and JEP 546 (adaptive heap sizing) for ZGC. Together they aim to make `-Xmx`, `-Xms` and `-XX:+AlwaysPreTouch` unnecessary. Here's why choosing a heap size has always been guesswork, how the new controller decides, and what it means for your containers.
2. **IBM and Red Hat say their Lightwell programme has found and fixed more than 400 previously unknown vulnerabilities in widely used Java libraries.** They haven't said which libraries. Here's what Lightwell is, why "backporting" is the part that matters, why Java codebases are especially exposed, and what you can do this week.
3. **On the radar:** 4 more things that happened, listed at the end.

STORY 1 ZGC learns to size its own heap

JDK · HOTSPOT GC · JEPS 545 AND 546 (CANDIDATE) · IMPLEMENTATION PR OPENED 7 OCT 2026 · ~20 MIN READ

Ask ten Java teams why their service runs with `-Xmx4g` and you'll get roughly ten versions of the same answer: "someone picked it once, and it hasn't fallen over." The heap size is the most important number you give the JVM, and almost nobody chooses it from evidence. Set it too low and the garbage collector works itself into the ground. Set it too high and you pay for memory you never use, or your container gets killed. On 7 October, the engineer who leads Oracle's work on ZGC submitted roughly 5,600 lines of HotSpot code that try to make the number unnecessary. The JVM would watch the machine and pick the heap size itself, and keep picking it again for as long as the program runs.

WHAT HAPPENED

On 7 October 2026, Erik Österlund (GitHub user `fisk`) opened pull request #33243 on the `openjdk/jdk` repository, titled *"Implement JEP 545: Faster Startup and Warmup with ZGC & JEP 546: Adaptive Heap Sizing for ZGC"*, and marked it ready for review the same day.[1] Axel Boldt-Christmas and Joel Sikström are credited as co-contributors.[1] The two commits we downloaded from the PR change 84 files, adding about 5,600 lines and removing about 500. Most of the new code sits in two new HotSpot source files: `zAdaptiveHeap.cpp`, the decision-maker, and `zMemoryWorker.cpp`, which does the background memory work.[2]

The PR's description summarises the two halves.[1] JEP 546 lets ZGC "adaptively adjust the size of the heap based on the resources available, the needs of the application, and the needs of other applications competing for those resources." JEP 545 makes ZGC acquire and prepare physical memory faster, so applications start and warm up sooner. The implementation "continuously monitors system, container, and JVM resource usage" and uses a **control system** ≈ *a feedback loop that measures what's happening, compares it with a goal, and adjusts a setting to close the gap, like a thermostat* to settle on a suitable heap size.

Both are **JEPs** ≈ *JDK Enhancement Proposals: the documents that describe a significant change to Java and track it from draft to delivery* at **Candidate** status. InfoQ reported their promotion from draft in its roundup of the week of 21 September.[3] Neither has been targeted to a release yet. The PR can't be merged until two reviewers approve it and a **CSR** ≈ *Compatibility and Specification Review: the OpenJDK process that signs off on any change to flags, APIs or behaviour that users can see* request, JDK-8380515, is approved. As of the PR page we read, no reviews had been posted and the CSR was still pending.[1] So this is a "how Java is about to change" story, not a "change your flags today" story. It's worth understanding now, because it changes one of the oldest habits in Java operations.

THE BACKGROUND YOU NEED

The heap, and why its size is a trade-off

Every object you create with `new` lives in the **heap** ≈ *the region of memory where the JVM stores objects; its size is bounded by -Xmx*. A **garbage collector** ≈ *the part of the JVM that finds objects your program can no longer reach and reclaims their memory* (GC) periodically works out which objects are still reachable and frees the rest.

The heap size controls a trade-off between memory and CPU. A small heap fills up quickly, so the GC runs often and burns CPU. A big heap means fewer collections but more memory reserved. Oracle's own ZGC tuning guide puts it plainly: "The most important tuning option for ZGC is setting the maximum heap size... it's all about finding a balance between memory usage and how often the GC needs to run."[4]

You control it with two flags: **-Xmx** ≈ *the maximum heap size; the heap may never grow beyond it* and **-Xms** ≈ *the initial (and minimum) heap size the JVM starts with*. If you set neither, the JVM's **ergonomics** ≈ *the rules the JVM uses to pick default settings from the machine it finds itself on* choose for you. On our sandbox (JDK 21, about 7 GB of RAM, two CPUs), `java -XX:+UseZGC -Xlog:gc+init -version` reported an initial capacity of 94 MB and a maximum of 1,496 MB. `-XX:+PrintFlagsFinal` showed why: `MaxRAMPercentage` defaults to 25, so the JVM caps the heap at a quarter of the machine's memory, whatever the application actually needs.

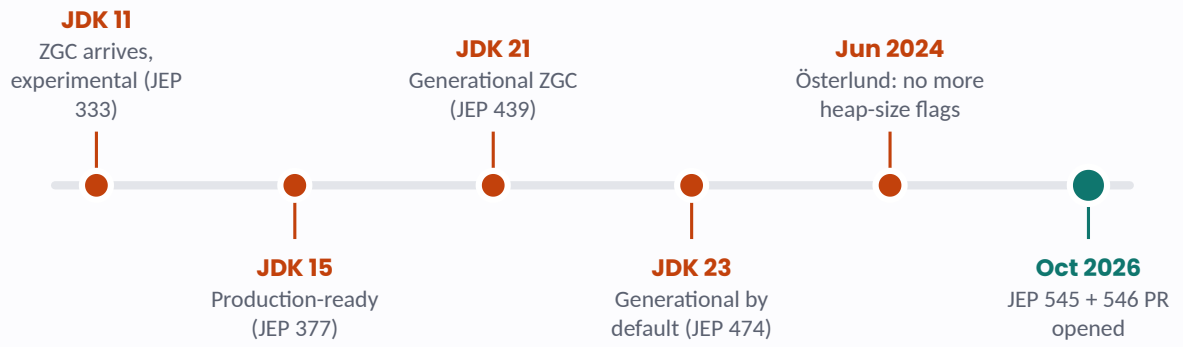
That 25% is the root of a familiar container story. Give a pod 2 GB and, by default, the heap can only grow to about 512 MB. So teams raise `MaxRAMPercentage` to 75 or 80. Then something outside the heap grows (thread stacks, direct buffers, the JIT's code cache), the container exceeds its limit, and the kernel's **OOM killer** ≈ *the Linux mechanism that terminates a process when a machine or container runs out of memory* ends the process with exit code 137. Neither number came from measuring anything.

ZGC in one page

ZGC ≈ *the Z Garbage Collector: a HotSpot collector designed to keep pauses under about a millisecond regardless of heap size* is HotSpot's low-latency collector. A **GC pause** ≈ *a moment when the JVM stops all application threads so the collector can do*

work safely in ZGC stays tiny because almost all of its work is **concurrent** *≈ done on GC threads while your application threads keep running*. ZGC arrived as an experimental feature in JEP 333 and has been "available for production use since JDK 15" through JEP 377.[5] JDK 21 added a **generational** *≈ splitting the heap into a young area for new objects and an old area for survivors, because most objects die young* mode (JEP 439),[5] JDK 23 made generational mode the default (JEP 474),[6] and JEP 490 later removed the old non-generational mode entirely.[6]

ZGC's road to self-sizing



ZGC's pauses don't depend on heap size, but how *often* it collects does. That makes the size question sharper for ZGC than for most collectors. ZGC copes with a heap that's too small by collecting constantly. If the application allocates faster than ZGC can reclaim, threads hit an **allocation stall** *≈ when an application thread must wait for the GC to free memory before it can create a new object; a latency spike*, which is exactly the latency spike you chose ZGC to avoid.

The knobs we have today, and why they're not enough

ZGC already has some elasticity. It can grow the heap from `-Xms` towards `-Xmx`, and by default it **uncommits** *≈ gives physical memory back to the operating system while keeping the address range reserved* memory that has gone unused for `ZUncommitDelay`, which is 300 seconds by default.[4] There's also **SoftMaxHeapSize** *≈ a soft ceiling on heap growth; ZGC exceeds it only to avoid an allocation stall*: with `-Xmx5g -XX:SoftMaxHeapSize=4g`, ZGC aims for 4 GB but may briefly use 5 GB.[4] But every one of these is still a number *you* choose up front.

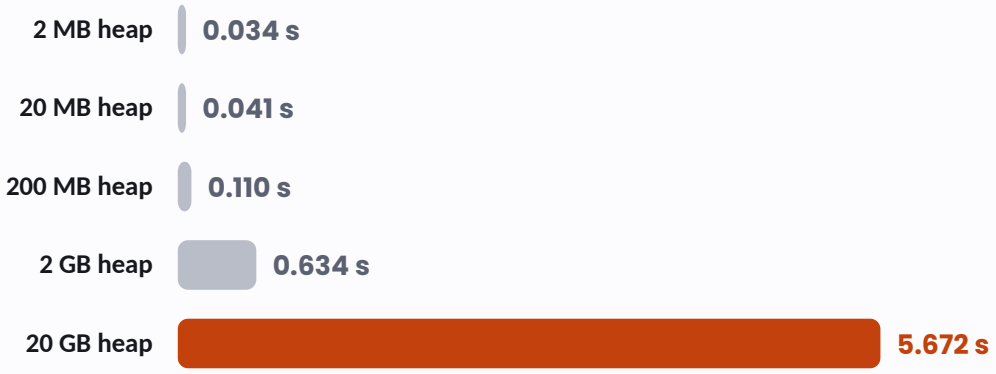
Then there's the classic low-latency recipe. Oracle's guide suggests that for "extremely low latency" you set `-Xms` equal to `-Xmx` and add `-XX:+AlwaysPreTouch`.[4] To see why, you need to know how the operating system hands out memory.

How memory actually reaches the JVM

When the JVM **commits** *≈ asks the operating system to back a range of addresses with real, physical memory* heap memory, Linux mostly just makes a promise. Physical RAM is assigned lazily, one **page** *≈ the unit the OS manages memory in; normally 4 KB on x86 Linux* at a time, the first time each page is written. Each first write triggers a **page fault** *≈ a trap into the kernel when a program touches memory that isn't mapped to physical RAM yet; the kernel then assigns a page*. Faults are cheap one at a time but expensive by the hundred thousand. If they happen while your service is taking its first wave of traffic, they show up as slow requests.

Pre-touching ≈ *writing to every page of the heap at startup so all the page faults happen before the application runs* moves that cost to startup. The trade-off is that startup gets slow, and it gets slower the bigger the heap. JEP 545's own measurements on JDK 25 show how quickly that grows:[7]

JDK 25 startup time with -XX:+AlwaysPreTouch, by heap size (-Xms = -Xmx)



Source: the JEP 545 text (openjdk.org/jeps/8329758). Every page of the heap is written once before main() runs, so startup grows with heap size.

So today you choose between two extremes. Start small and fast, and suffer slow warmup as the heap grows and page-faults under load. Or start big and pre-touched, and suffer slow startup plus memory you may never use. The JEP describes exactly this dilemma: users "typically pick one extreme."[7]

Large pages ≈ *2 MB memory pages (instead of 4 KB) that mean far fewer page-table entries and fewer TLB misses* help further. The **TLB** ≈ *Translation Lookaside Buffer: a small CPU cache of recent virtual-to-physical address translations; a miss costs a slow page-table walk* covers 512 times more memory per entry with a 2 MB page. Oracle's guide says large pages "generally yield better performance… and comes with no real disadvantage, except that it's slightly more complicated to setup", which usually means root access and kernel settings.[4] Most teams never bother.

Where the idea came from

None of this is new frustration. In June 2024, Jonathan Joo from Google proposed on the hotspot-dev mailing list to bring "Adaptable Heap Sizing" to G1. Google had been running it internally across many jobs, with an external thread that tuned two JVM flags about once a second based on GC CPU overhead and container limits.[8] Österlund replied that he was already working on automatic heap sizing for ZGC, and set out a bolder goal: "My vision is that users shouldn't have to set heap sizes." He wanted to ship the policy inside the JVM, not just hooks for a custom launcher: "the goal is to ship both the mechanisms and the policies needed."[9] He has since presented the work at the JVM Language Summit, in a talk titled "Automatic Heap Sizing in ZGC – Making Memory Management Effortless."[10] PR #33243 is that vision in code.

HOW IT WORKS

There are two JEPs because there are two separate problems. JEP 546 decides **how big** the heap should be. JEP 545 makes **getting there** cheap.

JEP 545: start tiny, grow fast, prepare memory in the background

The JEP 545 text lists three changes.[7]

1. **A 2 MB starting heap.** Today ZGC commits 0.2% of RAM at startup: 2 MB on a 1 GB machine, but 256 MB on a 128 GB machine. The proposal always starts at 2 MB.
2. **Rapid, exponential expansion.** ZGC treats frequent GC cycles as a sign that the heap is too small, and commits more memory asynchronously, doubling its way up so the number of expansions grows only logarithmically. It also estimates the application's normal allocation rate so it can expand *before* it's needed. The commit work runs on background worker threads, capped at **25% of CPU** so they never starve your application threads. The PR encodes that cap as `ZConcurrentWorkersCPUShare = 0.25`.[2]
3. **Concurrent pre-touching and automatic large pages.** Instead of pre-touching the whole heap at startup, worker threads pre-touch each new chunk as the heap grows. On Linux 6.1 and later, they also upgrade the new memory from 4 KB to 2 MB pages, with no kernel configuration needed.[7] In the code this uses the kernel's **MADV_COLLAPSE** *≈ a Linux madvise() hint (kernel 6.1+) asking the kernel to merge a range of small pages into huge pages right away*, and the GC log reports large pages as "Enabled (Transparent, JVM enforced)."[1,2] The PR calls this background work "heating" memory.

The JEP's case study runs Spring PetClinic on a 32 GB machine. After one health check, load ramps from 1,000 to 30,000 requests per second over 10 seconds, then holds for two minutes. "Warm" means the **P99** *≈ the 99th-percentile latency: 99% of requests are faster than this* response time stays under 20 ms.[7]

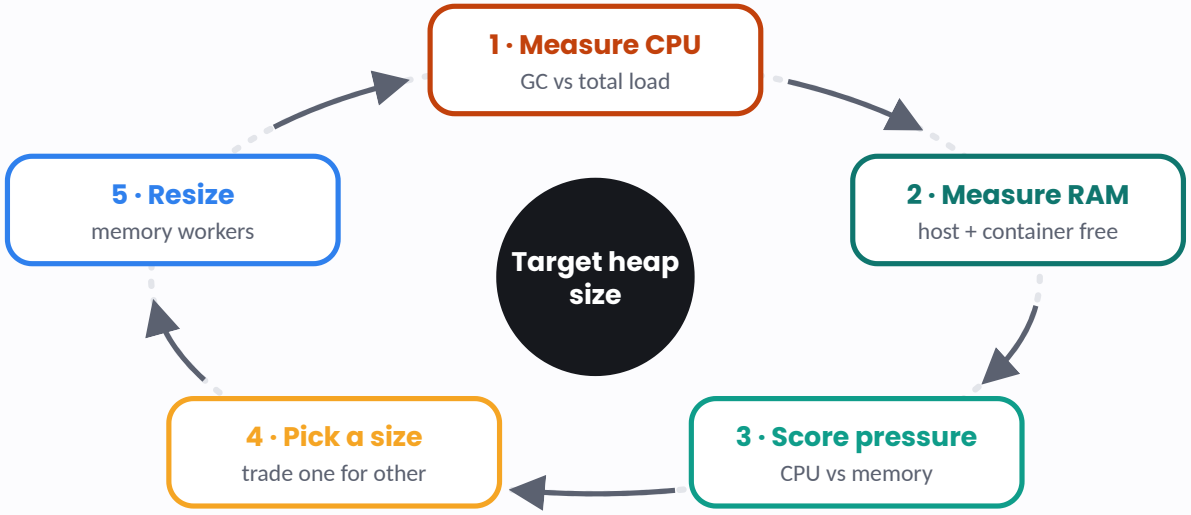
Configuration	Startup	Warm after
JDK 25, `-XX:+UseZGC -Xms2M -Xmx20G`	5.425 s	about 80 s
JDK 25, `-Xms20G -Xmx20G -XX:+AlwaysPreTouch`	9.009 s	about 7.8 s into the ramp
New, `-XX:+UseZGC` only	5.400 s	P99 under 20 ms throughout; peak reached 8.0 s into the ramp

The new behaviour gets the startup of the small heap and the warmup of the pre-touched one, from a command line with no sizing flags. The JEP summarises it as 38% lower startup time and 90% lower warmup time.[7] The ablations show that every part matters. Without concurrent pre-touching, warmup takes 34 seconds. Without large pages, it takes 18 seconds.[7]

JEP 546: a controller that weighs CPU against memory

Growing fast is only half the job. Something must decide how big is *enough*, and when to shrink again because a neighbour needs the RAM. That's JEP 546, and the code in `zAdaptiveHeap.cpp` shows how it decides.[2]

The adaptive heap controller, sampled continuously



It measures CPU pressure. For each young and old collection, the controller samples how much CPU time the GC used, how much the memory workers used, how busy the whole process is, and how busy the machine or container is. When it can read a container's CPU limit, it uses that instead of the host's core count.[2] It then asks a queueing-theory question. The code includes an implementation of the **Erlang C formula** *≈ a classic queueing-theory formula, from telephone exchanges, for the probability that a new arrival has to wait for one of c busy servers*, and uses it to estimate how much extra GC work on c CPUs would delay *your* threads at the 99th percentile.[2] In plain terms: on an idle 16-core box, a bit more GC is nearly free. On a 2-vCPU pod at 80% utilisation, every GC cycle steals time your requests needed.

It measures memory pressure. The controller reads how much memory the machine, and the container if there is one, has left. It works with three thresholds on the remaining reserve: it *prefers* to keep 20% free ("concerning"), *fights* to keep 10% free ("high"), and *enforces* 5% ("critical").[2] Between "concerning" and "high", memory pressure rises quadratically. Past "high", it rises exponentially, and the JVM gets increasingly willing to spend CPU on GC to give memory back. If memory use on the machine is unstable, or the OS is compressing memory, the controller widens the safety margin.[2]

It trades one against the other. The single dial you're offered is a new flag, **ZGCIntensity** *≈ a new ZGC flag (default 5 when adaptive sizing is on) that sets the balance between the CPU the GC uses and the memory the heap takes*. The `java` man-page text added in the PR says it "configures the trade-off between memory and CPU usage on the system," and that the default of 5 strikes "a balance between memory and CPU that is reasonable for most systems."[2] It's declared **manageable** *≈ a JVM flag that can be changed while the program runs, for example with jcmd*, so you can adjust it on a live process, although you can't use it to switch adaptive sizing on or off at runtime.[2]

Then it acts, in the background. The memory workers commit, uncommit and pre-touch memory to move the heap towards the target. The PR's own summary says they can "accelerate heap growth or shrinkage when urgent."[1]

Which flags still mean something

Here's the part that will matter to your deployment scripts. In the PR as submitted, adaptive sizing is **opt-in**: the new flag `-XX:+ZAdaptiveHeapSizing` defaults to `false`, and the man page says "By default, this option is disabled."[2] Whether it becomes the default, and when, is for the JEP and review process to settle. When you do turn it on, the code changes what the old flags mean:[2]

- **No sizing flags at all:** the GC log reports "Heap Sizing: Adaptive (Implicit Boundaries)." The minimum heap defaults to 2 MB. The *maximum* defaults to 100% of the **machine's** memory, not 25%. The comment in `zArguments.cpp` explains: "we might need to scale up to most of the underlying memory." The controller, not the cap, keeps the heap honest.
- **Only `-Xmx` (or only `-Xms`) set:** "Adaptive (Explicit Boundaries)." Your numbers become fences, and the controller works between them.
- **`-Xms` equal to `-Xmx`:** there's nothing to adapt, so the JVM turns adaptive sizing off. If you asked for it explicitly, it logs "Adaptive heap sizing was enabled, but heap size is fixed."
- **`SoftMaxHeapSize`:** today ZGC quietly sets it to 90% of `-Xmx` when you haven't set a limit. With adaptive sizing on, it defaults to "no soft max," and the controller's own target takes its place.

Running a ZGC service: today vs with JEPs 545/546

Today: you guess the numbers

```
java -XX:+UseZGC \
     -Xms4g -Xmx4g \
     -XX:+AlwaysPreTouch \
     -XX:SoftMaxHeapSize=3g \
     -jar orders-service.jar

# in a container, also:
#  -XX:MaxRAMPercentage=75
```

Proposed: the JVM decides

```
java -XX:+UseZGC \
     -XX:+ZAdaptiveHeapSizing \
     -jar orders-service.jar


# optional: shift the CPU vs
# memory balance at runtime
jcmd <pid> VM.set_flag \
     ZGCIntensity 3
```

Right-hand side uses flags from PR #33243 (not yet in any JDK build). In the PR, -XX:+ZAdaptiveHeapSizing is off by default; the JEP 545 text describes the goal of plain -XX:+UseZGC being enough. The value 3 is only an example: the PR's documentation doesn't say which direction favours memory, so test before relying on it.

WHAT IT MEANS FOR YOU

If you run ZGC in containers, this is aimed squarely at you. The controller reads container memory and CPU limits directly, which removes the guesswork around `MaxRAMPercentage`.[2] More interesting is what happens on a crowded host. Today, several JVMs on one node each grab a fixed slice of memory whether they need it or not. An adaptive JVM gives memory back when the machine runs short and takes more when it's free. That's the "needs of other applications competing for those resources" part of JEP 546.[1] It's the same economics Google described: recover memory from applications that were configured with far more than they use.[8]

If you run G1 (the default), nothing changes yet. This is ZGC-only. The idea isn't, though: Google's 2024 proposal was for G1, and the same mailing-list thread shows the HotSpot team discussing the problem across collectors.[8] Treat this as the first collector to get the feature, not the only one that ever will.

If you've tuned for a big initial heap, read the JEP's risk section. A fixed 2 MB start means applications "tuned for a larger initial heap may warm up more slowly", and the JEP's example is a 128 GB machine that defaults to a 256 MB starting heap on JDK 25. The authors say they assume deployers re-tune GC settings after major JVM upgrades, and that the change is a net positive overall.[7] If your SLO is measured in the first minute after a deploy, measure before you remove flags.

If you want to try it, you can't download a build yet. The code isn't merged into the JDK's main line, so it isn't in the JDK 28 early-access builds. Watch the PR and the JEP pages. When it lands, the experiment is cheap: start with `-Xlog:gc+init` to see which "Heap Sizing" mode you're in, then compare memory use and P99 latency with and without `-XX:+ZAdaptiveHeapSizing` under realistic load.

💥 Watch out for

- **Nothing is targeted yet.** JEPs 545 and 546 are Candidates. The PR needs two reviews and an approved CSR. Flag names and defaults can still change in review.[1]
- **"No -Xmx" doesn't mean "no ceiling".** With adaptive sizing and no flags, the hard maximum is 100% of the *machine's* memory, and the controller is what keeps usage down. If you depend on a hard cap (for example, on a host with no container limit), set `-Xmx` and run in "explicit boundaries" mode.[2]
- **The off-heap memory is still yours to manage.** The controller sizes the Java heap. Thread stacks, direct `ByteBuffer`s, Netty's pools and the code cache still count against your container limit.
- **Large pages are Linux-only.** The JEP says the techniques may differ by OS: large pages by default on Linux, but not on Windows. The automatic upgrade needs Linux 6.1 or later.[7]
- **Benchmarks are Oracle's.** The PetClinic numbers come from the JEP authors' own case study on one 32 GB machine. Your results will depend on allocation rate, core count and what else shares the host.

WHAT HAPPENS NEXT

The PR now needs review from other HotSpot GC engineers, plus approval of CSR JDK-8380515, before it can be merged.[1] The JEPs must also move from Candidate to *Proposed to Target* and then *Targeted* for a specific release. That would most likely be JDK 28, due in March 2027, if review completes in time for its feature freeze. The JEP lists two pieces of future work: balancing CPU between concurrent collection and concurrent memory commit, and benefiting from Project Leyden's ahead-of-time compilation, which would leave more CPU free for ZGC during startup.[7] We'll follow up when either JEP is targeted.

📖 Words from this story

Control system: a feedback loop that measures what's happening, compares it with a goal, and adjusts a setting to close the gap, like a thermostat.

JEP: JDK Enhancement Proposal, the document that describes a significant change to Java and tracks it from draft to delivery.

CSR: Compatibility and Specification Review, the OpenJDK process that signs off on any change to flags, APIs or behaviour that users can see.

Heap: the region of memory where the JVM stores objects; its size is bounded by -Xmx.

Garbage collector: the part of the JVM that finds objects your program can no longer reach and reclaims their memory.

-Xmx / -Xms: the maximum heap size, and the initial (and minimum) heap size.

Ergonomics: the rules the JVM uses to pick default settings from the machine it finds itself on.

OOM killer: the Linux mechanism that terminates a process when a machine or container runs out of memory.

ZGC: the Z Garbage Collector, a HotSpot collector designed to keep pauses under about a millisecond regardless of heap size.

GC pause: a moment when the JVM stops all application threads so the collector can do work safely.

Concurrent: done on GC threads while your application threads keep running.

Generational: splitting the heap into a young area for new objects and an old area for survivors, because most objects die young.

Allocation stall: when an application thread must wait for the GC to free memory before it can create a new object.

Uncommit: give physical memory back to the operating system while keeping the address range reserved.

SoftMaxHeapSize: a soft ceiling on heap growth; ZGC exceeds it only to avoid an allocation stall.

Commit: ask the operating system to back a range of addresses with real, physical memory.

Page: the unit the OS manages memory in; normally 4 KB on x86 Linux.

Page fault: a trap into the kernel when a program touches memory that isn't mapped to physical RAM yet.

Pre-touching: writing to every page of the heap at startup so all the page faults happen before the application runs.

Large pages: 2 MB memory pages that mean far fewer page-table entries and fewer TLB misses.

TLB: Translation Lookaside Buffer, a small CPU cache of recent address translations.

MADV_COLLAPSE: a Linux madvise() hint (kernel 6.1+) asking the kernel to merge small pages into huge pages right away.

P99: the 99th-percentile latency; 99% of requests are faster than this.

Erlang C formula: a queueing-theory formula for the probability that a new arrival has to wait for one of c busy servers.

ZGCIntensity: a new ZGC flag (default 5 when adaptive sizing is on) that sets the balance between the CPU the GC uses and the memory the heap takes.

Manageable flag: a JVM flag that can be changed while the program runs, for example with jcmd.

STORY 2 IBM and Red Hat find 400 holes in Java's libraries

SECURITY · OPEN-SOURCE SUPPLY CHAIN · ANNOUNCED 6 OCT 2026 · ~13 MIN READ

Open your service's `pom.xml` and count the dependencies. Now run `mvn dependency:tree` and count again. The second number is the one that matters, and it's usually far bigger than the first. Every one of those JARs is code you ship, run with full privileges, and mostly didn't read. On 6 October, IBM and Red Hat said that a team of engineers working with AI tools had gone looking in exactly that kind of code. They found more than 400 security vulnerabilities that nobody had reported before, all in widely used Java libraries. They haven't told anyone which libraries yet. Here's what we know, what we don't, and why the most interesting part of the announcement isn't the number.

WHAT HAPPENED

On 6 October 2026, IBM and Red Hat announced that **Lightwell**, their joint open-source security programme, has "found, fixed, and backported patches for more than 400 previously unknown bugs in widely deployed, production-grade software", specifically in widely used Java libraries.[11] On the same day they made **Lightwell Clearinghouse** generally available. It's a paid service where enterprise customers can submit specific open-source dependencies for priority review and repair, "including fixes for older software versions still in use."[11]

Gunnar Hellekson, Red Hat's vice president and general manager for Lightwell, framed it around AI: "AI agents shifted the threat landscape overnight, exploiting old dependencies at machine speed." He argued that attackers need only "one small crack" to chain together an attack.[11,12] And he made the point this story is really about: "Finding those bugs is only half the battle." The real work is "backporting fixes directly into active production apps so customers do not have to pick between security and uptime."[13]

The announcement leaves out a lot. It names no libraries, gives no **CVE** ≈ *Common Vulnerabilities and Exposures: the public catalogue that gives each known security flaw an ID such as CVE-2026-40478* numbers, offers no severity breakdown and doesn't say over what period the 400 were found.[14,15] Hardware Busters reported, citing Phoronix, that an embargoed draft of the release said "300-plus", meaning the count was still climbing while the release was being written.[16] None of this is unusual while fixes work their way through disclosure, but it means you can't yet check whether *you* are affected.

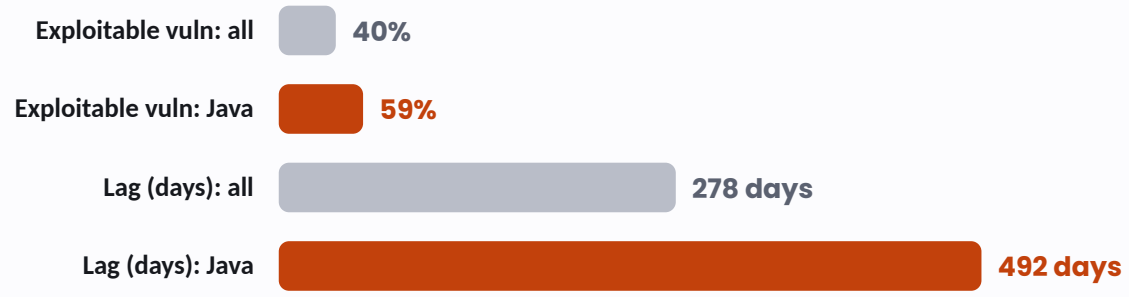
THE BACKGROUND YOU NEED

Your dependency tree is mostly other people's code

A **dependency** ≈ *a library your project declares it needs, e.g. in pom.xml or build.gradle* rarely comes alone. Add Spring Boot's web starter and you also get Spring MVC, an embedded Tomcat, Jackson and everything they need in turn. Those indirect libraries are **transitive dependencies** ≈ *libraries you get because one of your dependencies needs them, without declaring them yourself*, and they make up the bulk of most Java applications. You didn't choose their versions. Your build tool did, by resolving the tree.

That code is maintained by a mix of foundations, companies and volunteers, and it ages in place. Datadog's *2026 State of DevSecOps* report, published in February, put numbers on how badly that shows in Java:[17]

Java vs everyone: Datadog's 2026 State of DevSecOps



Top pair: share of services with at least one exploitable vulnerability. Bottom pair: median dependency lag. Source: Datadog, 2026 State of DevSecOps (26 Feb 2026). Lag = how far the median dependency is behind its latest major version. Java was highest on both measures. The two pairs use different units, so compare within each pair.

Java has the highest share of services with an exploitable vulnerability (59%, against 40% overall), and its median dependency is 492 days behind its latest major version, the furthest of any language measured. Datadog links the two directly: Java's dependencies "are typically the most outdated."[17] There's a reason. Java services live a long time, enterprise Java is conservative, and a major upgrade (Spring Boot 2 → 3, `javax` → `jakarta`, Jackson 2 → 3) can mean weeks of work. So teams stay on old versions, and old versions are exactly where unfixed bugs pile up.

Finding bugs is getting cheap

What changed in 2026 is the cost of finding bugs. When IBM and Red Hat launched the programme in May, they cited Anthropic's report that its Mythos Preview model had found nearly 3,900 high- or critical-severity vulnerabilities in open-source software.[19] InfoWorld's coverage of this week's announcement noted that Azul now offers a free JVM risk assessment explicitly pitched at the "blind spots" that AI-powered exploitation tools can find.[12] The same shift has a darker side for maintainers, who are being flooded with automated reports. We covered Spring's version of that problem, and its answer, a monthly "Patch Thursday" release train, in issue #2.

Lightwell's pitch is that discovery is no longer the bottleneck. Fixing is.

Why "just upgrade" doesn't work

When a library maintainer fixes a vulnerability, they usually ship the fix in their newest version, or in the newest version of each line they still support. If you're 492 days behind, that fix may be on the other side of a major-version upgrade, with API changes you'd have to absorb under time pressure.

The alternative is a **backport** *≈ taking a fix made in a newer version of some software and adapting it to apply cleanly to an older version*: someone takes the fix and re-applies it to the old version you actually run. Enterprise Linux distributions have done this for decades: they keep shipping a package under its old version number, with years of security fixes backported into it. That's Red Hat's core business, and Lightwell extends it to application libraries that Red Hat never shipped.

HOW IT WORKS

IBM and Red Hat have described Lightwell over three announcements, and the pieces fit together like this.

Lightwell in 2026

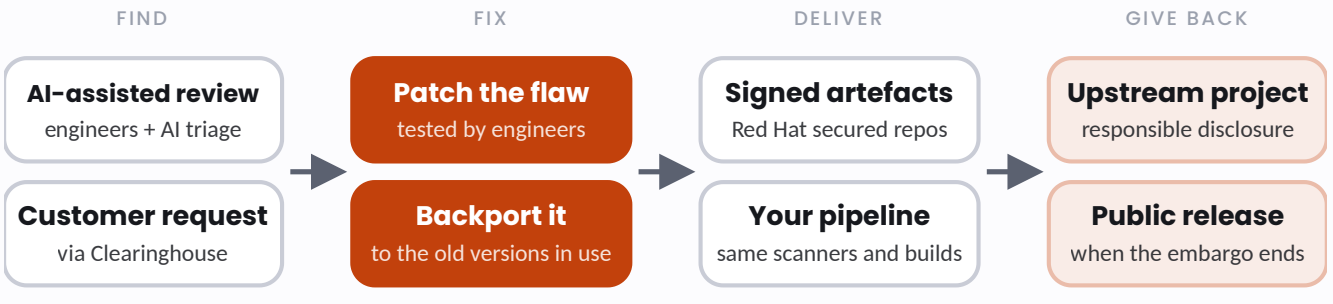


The commitment (May). On 28 May, IBM and Red Hat pledged $5 billion and "more than 20,000 engineers" to open-source security.[19] IBM said it uses more than 62,000 open-source packages and has deep expertise in over 10,000. The plan was to extend the patching and lifecycle work it already does for its own products to "independent libraries, language toolchains, AI frameworks, and data streaming platforms." Named areas included Linux, Java, Kubernetes and Kafka.[19] Early adopters listed were almost all large banks and payment companies, including Bank of America, Citi, Goldman Sachs, JPMorganChase, Mastercard and Visa.[19]

The catalogue (July). On 8 July, **Lightwell Network** became available as a subscription. It launched with a catalogue of more than 6,500 "remediated, digitally signed, and certified application-layer dependencies," covering Java and Python. It delivers signed binaries, source code and compliance artefacts, including an SBOM ≈ *Software Bill of Materials: a machine-readable list of every component and version inside a piece of software*, "into existing pipelines." It "backports critical fixes to specific long-lived production versions, so teams can avoid major upstream upgrades."[20]

The milestone and the clearinghouse (October). This week's announcement adds the 400-plus new findings and opens Clearinghouse to all eligible enterprises. Red Hat's product page describes Clearinghouse as everything in Network plus "customer-specific vulnerability and package requests," vulnerability verification, disclosure coordination and a dedicated security account manager.[21] Pricing isn't published. It's an annual subscription, and you contact sales.[21]

How a Lightwell fix travels



The last column is where the open-source community's interests come in. Fixes that apply upstream are contributed back to the original projects "under responsible disclosure protocols," with "embargo protections" for Clearinghouse participants.[11] **Responsible disclosure** ≈ *reporting a vulnerability privately to the maintainers and giving them time to fix it before details are published* means the maintainers hear first. An **embargo** ≈ *an agreed period during which a vulnerability is kept secret while fixes are prepared* means paying customers can have a patched JAR before the flaw is public. Everyone else gets the fix from the public upstream release, "whenever disclosure allows."[14] That's why there's no list of libraries. Publishing one now would tell attackers where to look before most users can patch.

Delivery deliberately looks boring. Patched artefacts come from Red Hat's secured repositories, which you add alongside the public ones you already use, so your existing scanners, repositories, pipelines and tests don't change.[11,21] Neither the press release nor the product page specifies the artefact format or version naming (for example, whether a backported Maven artefact gets a suffix). Ask before you assume.

WHAT IT MEANS FOR YOU

Expect a wave of Java library advisories. As fixes move upstream and embargoes end, "previously unknown" becomes "published CVE." You don't know which libraries are involved, but you can make sure you'll notice quickly. That means a **software composition analysis** ≈ *SCA: tools such as OWASP Dependency-Check, Snyk or Dependabot that match your dependency tree against vulnerability databases* scan in CI that fails or alerts on new findings, plus a dependency inventory you can query.

Know how to override a transitive version. When a fix lands in a library you don't declare yourself, you need to force the patched version without waiting for your framework's next release. Here's what that looked like for a real case from this year: CVE-2026-40478, a critical (CVSS 9.1) template-injection sandbox bypass in Thymeleaf, fixed in 3.1.4.RELEASE.[22] (This flaw was reported in April, before Lightwell's announcements, and isn't one of the 400. It's simply a recent, well-documented example.)

Forcing a patched transitive dependency

Maven: pom.xml

```xml
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.thymeleaf</groupId>
      <artifactId>thymeleaf</artifactId>
      <version>3.1.4.RELEASE</version>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Gradle: build.gradle.kts

```
dependencies {
  constraints {
    implementation(
      "org.thymeleaf:thymeleaf:3.1.4.RELEASE"
    ) {
      because("CVE-2026-40478")
    }
  }
}
```

Fixed version from the CSO Online report on CVE-2026-40478. In a Spring Boot project you can usually set the thymeleaf.version property instead. Then verify with mvn dependency:tree or gradle dependencies.

🌍 In the real world: what a library flaw looks like in a Spring app

CVE-2026-40478 shows why "it's just a library" isn't reassuring. Thymeleaf blocked dangerous expressions partly by searching for the keyword `new` followed by a space, but only an ASCII space. The **SpEL** ≈ *Spring Expression Language: the expression language Spring uses in annotations, configuration and, via Thymeleaf, templates* parser also accepts a tab. Thymeleaf also blocked `java.*` classes but not `org.springframework.*`. So Endor Labs' proof of concept used a tab after `new` to create a `FileSystemResource` and write a file, and the researchers said that could escalate to remote code execution. Every Thymeleaf version before 3.1.4.RELEASE was affected, with no workaround.[22] That's a **server-side template injection** ≈ *SSTI: an attack where user input gets evaluated as template code on the server* flaw in the de facto template engine for Spring MVC. It's the kind of bug Lightwell says it's hunting, multiplied by 400.

Should you buy Lightwell? That's a business decision this newsletter can't make for you. It is aimed at large regulated enterprises: its named early adopters were banks and payment companies, and its first Clearinghouse tier was limited to financial services.[20] If you're a smaller team, the free path still works. Upstream fixes reach public releases once they're disclosed, and most of the protection comes from being able to upgrade quickly when they do. Notice that Datadog's 492-day lag is Lightwell's market. A team that keeps its dependencies within a few months of current mostly doesn't need anyone to backport fixes for it.

Use the quiet period. Run `mvn dependency:tree` (or `gradle dependencies`) on your most important service and look for libraries more than one major version behind, or libraries that haven't had a release in years. Datadog found 42% of services rely on libraries that are no longer actively maintained.[18] Those are the ones where no upstream fix will ever come, and where a commercial backport, or replacing the library, is the only option.

💥 Watch out for

- **The 400 can't be checked yet.** No libraries, CVEs or severities have been published. Some findings may turn out to be low severity. Treat the number as a claim until the advisories appear.[14,16]
- **Backported artefacts are a fork.** A patched old version from a vendor isn't the same artefact as upstream. Know which repository each JAR came from, and make sure your SBOM records it.
- **AI-found bugs cut both ways.** Hardware Busters points out a tension the press release skips: the same AI tools have flooded volunteer maintainers with low-quality automated reports. IBM and Red Hat say theirs are different because each comes with a tested patch. Whether maintainers agree can't be judged until the projects are named.[16]
- **Don't stop upgrading.** A backport buys time. It doesn't remove the need to move to supported versions eventually.

WHAT HAPPENS NEXT

Watch the security advisories of the big Java libraries over the coming weeks. As fixes move upstream and embargoes end, the 400 should start to appear as CVEs with names attached. Red Hat says it is "just getting started," and IBM's July plan talks about growing the catalogue "from thousands to millions of packages."[11,20] Closer to home, Oracle's next Critical Patch Update is due on 20 October and the first Spring Patch Thursday release train on 22 October (both covered in issues #1 and #2). It's a good fortnight to make sure your upgrade path is short.

📖 Words from this story

CVE: Common Vulnerabilities and Exposures, the public catalogue that gives each known security flaw an ID.

Dependency: a library your project declares it needs, e.g. in pom.xml or build.gradle.

Transitive dependencies: libraries you get because one of your dependencies needs them, without declaring them yourself.

Backport: taking a fix made in a newer version of some software and adapting it to apply cleanly to an older version.

SBOM: Software Bill of Materials, a machine-readable list of every component and version inside a piece of software.

Responsible disclosure: reporting a vulnerability privately to the maintainers and giving them time to fix it before details are published.

Embargo: an agreed period during which a vulnerability is kept secret while fixes are prepared.

Software composition analysis (SCA): tools such as OWASP Dependency-Check, Snyk or Dependabot that match your dependency tree against vulnerability databases.

SpEL: Spring Expression Language, used in annotations, configuration and, via Thymeleaf, templates.

Server-side template injection (SSTI): an attack where user input gets evaluated as template code on the server.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Gradle 9.8.1:** the first patch for 9.8 (7 Oct) fixes three high-rated vulnerabilities, including two where the Gradle daemon deserialised untrusted Java objects on unauthenticated channels, so upgrade your wrapper.[23]
- **"Java – The Year of the Runtime!":** Ron Pressler's Devoxx Belgium keynote (posted 11 Oct) argues that as AI agents make syntax matter less, the JVM's performance and operability matter more.[24]
- **Aspire 13.6 runs Java:** Microsoft's Aspire adds a prerelease `Aspire.Hosting.Java` package that can orchestrate executable JARs, Maven and Gradle wrappers, Spring Boot and Quarkus apps.[25]
- **JDK 28 build 19:** the `jdk-28+19` tag landed on 8 October, a week after build 18.[26]

1. **8380289: Implement JEP 545: Faster Startup and Warmup with ZGC & JEP 546: Adaptive Heap Sizing for ZGC, PR #33243**, Erik Österlund, openjdk/jdk on GitHub, opened 7 Oct 2026 (read 12 Oct).
2. **Source code of PR #33243** (commits a108cb4c and 7d2af28f, 7 Oct 2026), mainly zAdaptiveHeap, zArguments, zDirector, zGlobals and the java man page; read via git.
3. **Java News Roundup: TornadoVM 7.0, Groovy 6.0, GraalVM, Hibernate, Quarkus, Gradle, Maven**, InfoQ, 28 Sep 2026.
4. **HotSpot GC Tuning Guide: The Z Garbage Collector**, Oracle, JDK 23 documentation.
5. **JEP 439: Generational ZGC**, OpenJDK.
6. **JEP 474: ZGC: Generational Mode by Default** and **JEP 490: ZGC: Remove the Non-Generational Mode**, OpenJDK.
7. **JEP draft 8329758: Faster Startup and Warmup with ZGC** (now JEP 545), Erik Österlund, OpenJDK.

8. **Adaptable Heap Sizing for G1 GC**, Jonathan Joo and Erik Österlund, hotspot-dev mailing list, 13 Jun 2024.
9. **Re: Adaptable Heap Sizing for G1 GC**, hotspot-dev mailing list, 14 Jun 2024.
10. **Automatic Heap Sizing in ZGC – Making Memory Management Effortless**, Erik Österlund, JVM Language Summit (talk listing).
11. **IBM and Red Hat Remediate More Than 400 Previously Unknown Open Source Vulnerabilities**, Red Hat press release, 6 Oct 2026.
12. **Lightwell project filters out 400 Java library vulnerabilities**, Maxwell Cooter, InfoWorld, 9 Oct 2026.
13. **IBM's AI-powered vulnerability clearinghouse finds hundreds of Java flaws**, Eric Geller, Cybersecurity Dive, 6 Oct 2026.
14. **Java library vulnerabilities: IBM and Red Hat fix 400+ previously unknown flaws**, Anamarija Pogorelec, Help Net Security, 8 Oct 2026.
15. **IBM and Red Hat patch 400-plus unknown Java flaws, open Clearinghouse for fix requests**, Duncan Riley, SiliconANGLE, 6 Oct 2026.
16. **IBM and Red Hat Patch 400 Unknown Java Vulnerabilities, and the Hard Part Wasn't Finding Them**, Hardware Busters, 6 Oct 2026.
17. **State of DevSecOps 2026**, Datadog, 26 Feb 2026.
18. **87% of Organizations Are Running Software With Known, Exploitable Vulnerabilities, Datadog Finds**, Datadog press release, 26 Feb 2026.
19. **IBM and Red Hat Commit $5 Billion to Redefine the Future of Open Source in the AI Era**, IBM Newsroom, 28 May 2026.
20. **IBM and Red Hat Expand Lightwell with New Offerings to Build the Trust Infrastructure for AI-Era Open Source**, IBM Newsroom, 8 Jul 2026.
21. **Lightwell**, Red Hat product page, accessed 12 Oct 2026.
22. **Critical sandbox bypass fixed in popular Thymeleaf Java template engine**, Lucian Constantin, CSO Online, 17 Apr 2026.
23. **Gradle 9.8.1 release notes (tag v9.8.1)**, Gradle, GitHub, 7 Oct 2026.
24. **Java - The Year of the Runtime!**, Ron Pressler, Inside Java (Devoxx Belgium 2026), 11 Oct 2026.
25. **Aspire 13.6 Adds Persistent Dashboard Telemetry and First-Party Java and Rust Hosting**, Almir Vuk, InfoQ, 5 Oct 2026.
26. **openjdk/jdk tag jdk-28+19**, OpenJDK, GitHub, 8 Oct 2026.